UNIVERSIDADE CRUZEIRO DO SUL
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

PEDRO CAINAN RODRIGUES DE PAULA (RGM 41873891)
HENRIQUE DE SOUSA CUNHA (RGM 42873134)
CAUÃ DE LUCCA UEMA (RGM 42057019)
JESÚS ALEJANDRO ROMERO RIVERO (RGM 42772915)

CÓDIGO VERSIONADO, BANCO DE DADOS E INSTRUÇÕES DE EXECUÇÃO DO SISTEMA DITEC

Apêndice E — Entregável 5 da disciplina Análise e Projeto de Sistemas II

São Paulo
2026

PEDRO CAINAN RODRIGUES DE PAULA (RGM 41873891)
HENRIQUE DE SOUSA CUNHA (RGM 42873134)
CAUÃ DE LUCCA UEMA (RGM 42057019)
JESÚS ALEJANDRO ROMERO RIVERO (RGM 42772915)

CÓDIGO VERSIONADO, BANCO DE DADOS E INSTRUÇÕES DE EXECUÇÃO DO SISTEMA DITEC

Apêndice E — Entregável 5 da disciplina Análise e Projeto de Sistemas II

Documento técnico de implementação e execução, apresentado para a disciplina Análise e Projeto de Sistemas II, do curso Análise e Desenvolvimento de Sistemas, da Universidade Cruzeiro do Sul, como parte da avaliação do projeto da disciplina. Professor da disciplina: Silvio Rogerio de Lucia.

São Paulo
2026

RESUMO

Este documento descreve o código versionado, o banco de dados e as instruções para executar o sistema DITEC Assistência Técnica, na versão e58a3d2 de 9 de novembro de 2026. São apresentados a estrutura do repositório e do back-end, as tecnologias e suas versões, o catálogo das 42 rotas da API, os pré-requisitos, o passo a passo de execução local (banco de dados, variáveis de ambiente, back-end, front-end e testes), a implantação em produção no Render e no GitHub Pages, as migrações do banco com a pré-checagem e o roteiro de reversão corrigidos, uma lista de verificação da configuração de produção e uma proposta de integração contínua. As variáveis de ambiente são descritas sem valores, e nenhuma credencial é reproduzida. Os comandos são os do próprio repositório; os que não puderam ser executados no ambiente de elaboração deste documento são indicados.

Palavras-chave: instruções de execução; banco de dados; Flyway; Docker; implantação; integração contínua.

LISTA DE TABELAS

Tabela 1 – Estrutura do repositório 8
Tabela 2 – Pacotes do back-end e responsabilidades 8
Tabela 3 – Tecnologias e versões 9
Tabela 4 – Rotas da API por controller 10
Tabela 5 – Variáveis de ambiente do back-end 12
Tabela 6 – Lista de verificação da configuração de produção 17
Tabela 7 – Problemas comuns na execução e verificações sugeridas 20

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
CI	Integração contínua
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
CSV	Comma-Separated Values
DDL	Data Definition Language
HTTP	Hypertext Transfer Protocol
IA	Inteligência artificial
JPA	Jakarta Persistence API
JWT	JSON Web Token
OS	Ordem de serviço
REST	Representational State Transfer

SUMÁRIO

1 INTRODUÇÃO **7**
1.1 Finalidade **7**
1.2 Limites e convenções **7**
2 CÓDIGO VERSIONADO **8**
2.1 Estrutura do repositório **8**
2.2 Estrutura do back-end **8**
2.3 Tecnologias e versões **9**
2.4 Catálogo da API **10**
3 PRÉ-REQUISITOS **11**
4 EXECUÇÃO LOCAL **12**
4.1 Passo 1: obter o código **12**
4.2 Passo 2: criar o banco de dados e o usuário **12**
4.3 Passo 3: configurar as variáveis de ambiente **12**
4.4 Passo 4: iniciar o back-end **13**
4.5 Passo 5: iniciar o front-end **14**
4.6 Passo 6: executar os testes automatizados **14**
4.7 Execução em contêiner **14**
5 MIGRAÇÕES DO BANCO DE DADOS **15**
5.1 Antes de aplicar a V2 em um banco com dados **15**
5.2 Roteiro de reversão da V2 corrigido **15**
6 IMPLANTAÇÃO EM PRODUÇÃO **17**
6.1 Arquitetura de implantação **17**
6.2 Publicação do front-end **17**
6.3 Publicação da API **17**
6.4 Lista de verificação da configuração de produção **17**
7 INTEGRAÇÃO CONTÍNUA (PROPOSTA) **19**
8 SOLUÇÃO DE PROBLEMAS **20**
REFERÊNCIAS **21**

1 INTRODUÇÃO

1.1 Finalidade

Este documento é o quinto entregável sugerido pela disciplina: o código versionado, o banco de dados e as instruções para executar o projeto (PROJETO E DESENVOLVIMENTO DE SOFTWARE, [s. d.]). O objetivo é que uma pessoa que não participou do desenvolvimento consiga obter o código, preparar o banco, configurar o ambiente, executar o sistema e verificar que ele funciona.

1.2 Limites e convenções

As informações foram extraídas do repositório na versão e58a3d2 (EQUIPE DITEC, 2026a). Os comandos reproduzem o que o próprio repositório documenta (README, `run-local.sh`, `Dockerfile` e `deploy-pages.yml`). **Não foi possível executar `mvn test` nem a compilação do back-end no ambiente de elaboração deste documento**, que não alcançava o repositório central de dependências do Maven; a execução dos testes é, portanto, uma pendência (P02 do Apêndice D), e os resultados citados no Apêndice F são os relatados pelo grupo. O serviço publicado no Render também não pôde ser consultado, de modo que a configuração de produção é apresentada como lista de verificação (seção 6), e não como fato comprovado.

Nenhum valor de credencial (senha de banco, segredo JWT, chave de IA) aparece neste documento: as variáveis de ambiente são descritas por nome e finalidade, e os exemplos usam marcadores entre sinais de menor e maior, como .

2 CÓDIGO VERSIONADO

2.1 Estrutura do repositório

O repositório do projeto reúne o front-end, o back-end, os arquivos do banco de dados, a automação de testes de qualidade e a documentação, conforme a Tabela 1.

Tabela 1 – Estrutura do repositório

Pasta ou arquivo	Conteúdo
`Código-fonte/`	Front-end estático: `index.html`, `conta.html`, `tecnico.html`, `admin.html`, `styles.css`, scripts (`script.js`, `conta.js`, `tecnico.js`, `admin.js`, `dashboard-shared.js`, `os-equipe.js`, `config.js`), imagens e documentos do front-end (`README.md`, `SECURITY.md`)
`Prototipos/backend/`	Back-end Spring Boot: `pom.xml`, `Dockerfile`, `run-local.sh`, `.env.example` e o código em `src/main` e `src/test`
`Banco de Dados/database/`	Cópias de leitura do esquema (`schema.sql`, `V2__rastreamento_os.sql`), o script de criação do banco e do usuário e o README do banco
`.github/workflows/`	`deploy-pages.yml`, que publica o front-end no GitHub Pages
`qa/`	Scripts de qualidade do grupo (acessibilidade, teclado, interface, Lighthouse, fluxo da OS e latência do chat) e seus relatórios
`docs/`	Documentação do projeto, inclusive o relatório de correções do grupo
`README.md`	Apresentação do projeto

Fonte: Elaborado pelos autores (2026), a partir do repositório (EQUIPE DITEC, 2026a).

As migrações que o Flyway executa estão em `Prototipos/backend/src/main/resources/db/migration/` (`V1__schema.sql` e `V2__rastreamento_os.sql`). Os arquivos da pasta `Banco de Dados/database/` são cópias para leitura; **a fonte de verdade é a pasta de migrações do back-end**.

2.2 Estrutura do back-end

O pacote principal `com.ditec.assistencia` organiza o código em camadas, conforme a Tabela 2. Os números correspondem aos arquivos Java de produção; os 10 arquivos de teste ficam em `src/test`.

Tabela 2 – Pacotes do back-end e responsabilidades

Pacote	Arquivos	Responsabilidade
`controller`	7	Rotas REST: `AuthController`, `ClienteController`, `AgendamentoController`, `OrdemServicoController`, `ServicoController`, `ChatController`, `AdminController`
`service`	15	Regras de negócio, mapeadores, relatórios CSV e, no subpacote `ia`, o cliente da Gemini API

Pacote	Arquivos	Responsabilidade
`repository`	9	Acesso a dados (Spring Data JPA)
`entity`	9	Entidades persistentes, que correspondem às tabelas
`enums`	5	Enumerações, entre elas `StatusOS`, que concentra a matriz de transições
`security`	6	Filtros de JWT e de limite de requisições, utilitário de token e detalhes do usuário
`config`	4	Segurança, CORS, propriedades da IA e cliente HTTP
`exception`	7	Exceções de domínio e tratamento global de erros
`seed`	1	`DataSeeder`, que cria dados de demonstração em desenvolvimento
`dto`	—	Objetos de requisição e resposta

Fonte: Elaborado pelos autores (2026), a partir do código-fonte (EQUIPE DITEC, 2026a).

2.3 Tecnologias e versões

A Tabela 3 relaciona as tecnologias do projeto e as versões declaradas no repositório.

Tabela 3 – Tecnologias e versões

Componente	Tecnologia	Versão ou observação
Linguagem	Java	21 (`pom.xml` e `Dockerfile`)
Framework	Spring Boot	3.5.16 (`spring-boot-starter-parent`)
Web, persistência e validação	Spring Web, Spring Data JPA e Bean Validation	Gerenciadas pelo Spring Boot
Segurança	Spring Security e JJWT	JJWT 0.12.7; senhas com BCrypt
Banco de dados	MySQL	8.x (mínimo 8.0.16, por causa do `DROP CHECK` da migração V2)
Migrações	Flyway (`flyway-core` e `flyway-mysql`)	Gerenciado pelo Spring Boot
Monitoramento	Spring Boot Actuator	Somente `health` exposto
Testes	JUnit 5, Spring Boot Test, Spring Security Test e H2	Testes de integração com MockMvc e banco em memória
Contêiner	Docker	Estágio de compilação com Maven 3.9.11 e Temurin 21; execução com a imagem JRE 21
Front-end	HTML, CSS e JavaScript	Sem framework e sem Node para executar
Hospedagem	Render (API) e GitHub Pages (front-end)	Planos gratuitos
IA do chatbot	Google Gemini API	Modelo padrão `gemini-3.5-flash-lite`, alternativo `gemini-2.5-flash-lite`

Fonte: Elaborado pelos autores (2026), a partir do `pom.xml`, do `Dockerfile` e das configurações (EQUIPE DITEC, 2026a).

2.4 Catálogo da API

A API expõe 42 rotas, sob o prefixo `/api`, mais a rota de saúde `/actuator/health`. A Tabela 4 as agrupa por controller; o detalhe de cada rota, com perfil, requisitos, regras e testes, está na matriz de rastreabilidade do Apêndice F, e os critérios de resposta (códigos HTTP), no Apêndice B.

Tabela 4 – Rotas da API por controller

Controller	Rotas	Perfil de acesso	Requisitos
`AuthController`	3 (`/api/auth/register`, `login`, `me`)	Público (cadastro e entrada); autenticado (`me`)	RF01 a RF03
`ClienteController`	3 (`/api/clientes/me` e agendamentos)	Cliente	RF04, RF08
`AgendamentoController`	5 (`/api/agendamentos`)	Público (disponibilidade e criação); dono ou administrador (consulta, alteração e cancelamento)	RF05 a RF07
`OrdemServicoController`	10 (`/api/ordens-servico`)	Público (rastreio); cliente, técnico e administrador conforme a rota	RF09 a RF13, RF24 a RF27, RF29
`ServicoController`	1 (`/api/servicos`)	Público	RF14
`ChatController`	1 (`/api/chat`)	Público, com limite de requisições	RF15
`AdminController`	19 (`/api/admin`)	Administrador	RF16 a RF22, RF28

Fonte: Elaborado pelos autores (2026), a partir dos controllers (EQUIPE DITEC, 2026a).

3 PRÉ-REQUISITOS

Para executar o sistema localmente são necessários: Java 21 (JDK); Maven 3.9 ou superior; MySQL 8.0.16 ou superior; um navegador moderno; e um servidor estático para o front-end (por exemplo, a extensão Live Server do VS Code), pois o front-end é HTML, CSS e JavaScript e não requer Node. A chave da Google Gemini API é **opcional**: sem ela, o chatbot responde em modo automático, com texto de perguntas frequentes. Para executar em contêiner, é necessário o Docker.

4 EXECUÇÃO LOCAL

4.1 Passo 1: obter o código

```
git clone https://github.com/wikimiwi/DITEC-Landing-Page-main--v2.git
cd DITEC-Landing-Page-main--v2
```

4.2 Passo 2: criar o banco de dados e o usuário

Com o MySQL em execução, o script do repositório cria o banco `ditec_assistencia` (conjunto de caracteres `utf8mb4`) e o usuário de acesso. **Antes de executar, abra o script e troque a senha de exemplo por uma senha própria**, que deverá ser a mesma da variável `DB_PASSWORD`.

```
mysql -u root -p < "Banco de Dados/database/00-criar-banco-e-usuario.sql"
```

Não é necessário executar `schema.sql`: o Flyway cria as tabelas na primeira execução do back-end.

4.3 Passo 3: configurar as variáveis de ambiente

```
cd Prototipos/backend
cp .env.example .env
# edite o arquivo .env e preencha, no mínimo, DB_PASSWORD e JWT_SECRET
```

O arquivo `.env` não é versionado (consta do `.gitignore`). O segredo do JWT deve ter no mínimo 32 caracteres e pode ser gerado com `openssl rand -base64 48`. A Tabela 5 descreve as variáveis; os valores não são reproduzidos.

Tabela 5 – Variáveis de ambiente do back-end

Variável	Obrigatória	Padrão (quando há)	Finalidade
`DITEC_PROFILE`	Não	`dev`	Perfil ativo: `dev` (local) ou `prod` (hospedagem)
`DB_URL`	Não	MySQL local, banco `ditec_assistencia`	URL JDBC do banco
`DB_USERNAME`	Não	`ditec_user`	Usuário do banco
`DB_PASSWORD`	**Sim**	Nenhum	Senha do banco; a aplicação não inicia sem ela
`JWT_SECRET`	**Sim**	Nenhum	Segredo de assinatura do token (mínimo 32 caracteres)
`JWT_EXPIRATION_MINUTES`	Não	120	Validade do token, em minutos
`FRONTEND_URL`	Em produção	Origens locais (portas 5500 e 5501)	URL pública do front-end, liberada no CORS

Variável	Obrigatória	Padrão (quando há)	Finalidade
`DITEC_ALLOWED_ORIGINS`	Não	—	Lista de origens separadas por vírgula; tem prioridade sobre `FRONTEND_URL`
`GEMINI_API_KEY`	Não	Vazia	Chave da Gemini API; vazia, o chatbot usa respostas automáticas
`GEMINI_MODEL`	Não	`gemini-3.5-flash-lite`	Modelo principal da IA
`GEMINI_FALLBACK_MODEL`	Não	`gemini-2.5-flash-lite`	Modelo alternativo
`GEMINI_API_URL`, `GEMINI_READ_TIMEOUT_MS` e outras `GEMINI_*`	Não	Ver `application.yml`	Endereço, tempos limite e parâmetros de geração
`DITEC_SEED_ENABLED`	Não	`true`	Cria dados de demonstração quando não há administrador (desativado no perfil `prod`)
`DITEC_RATELIMIT_ENABLED`	Não	`true`	Limite de requisições por origem e rota
`DITEC_TRUST_PROXY`	Não	`false` (padrão), `true` no perfil `prod`	Considera o cabeçalho `X-Forwarded-For`; só deve ser verdadeiro atrás de proxy confiável
`PORT` e `JAVA_OPTS`	Não	`PORT` 8080	Usadas pelo `Dockerfile` (porta e opções da JVM)

Fonte: Elaborado pelos autores (2026), a partir de `.env.example`, `application.yml`, `application-prod.yml` e `Dockerfile` (EQUIPE DITEC, 2026a).

4.4 Passo 4: iniciar o back-end

A forma mais simples é o script do repositório, que carrega o `.env`, compila sem executar os testes e inicia uma cópia do arquivo JAR:

```
./run-local.sh
```

Alternativamente, com as variáveis já exportadas no ambiente:

```
mvn spring-boot:run
```

Na primeira execução, o Flyway aplica as migrações V1 e V2 e, no perfil de desenvolvimento, o `DataSeeder` cria dados de demonstração (administrador, técnico, cliente, serviços, agendamento e OS de exemplo) se ainda não houver administrador. As credenciais das contas de demonstração estão no README do projeto e não são reproduzidas aqui. O back-end fica disponível em `http://localhost:8080`.

Para verificar, a rota de saúde deve responder com o estado `UP`:

```
curl http://localhost:8080/actuator/health
```

4.5 Passo 5: iniciar o front-end

O front-end deve ser servido por um servidor estático, e não aberto como arquivo, para que as chamadas à API e o CORS funcionem. Com o Live Server, abra a pasta `Código-fonte` e inicie o servidor na porta 5500 ou 5501, que são as origens liberadas por padrão no CORS. O arquivo `config.js` detecta o ambiente: em `localhost` ou `127.0.0.1`, a API é `http://localhost:8080`; em qualquer outro endereço, é a API publicada no Render. Qualquer outro servidor estático pode ser usado, desde que sua origem seja informada em `DITEC_ALLOWED_ORIGINS`.

4.6 Passo 6: executar os testes automatizados

```
cd Prototipos/backend
mvn test
```

O Maven executa as classes `*Test`, `*Tests` e `*IT` (configuração do `maven-surefire-plugin` no `pom.xml`), com banco H2 em memória e o perfil de teste, que desliga o limite de requisições. O grupo relata 52 testes aprovados (Apêndice F). Esta execução **não foi reproduzida** pelos autores deste documento, por falta de acesso ao repositório de dependências.

4.7 Execução em contêiner

O `Dockerfile` compila o projeto em um estágio com Maven e Java 21 e executa o JAR em uma imagem JRE 21, na porta indicada por `PORT` (padrão 8080). As variáveis da Tabela 5 são fornecidas no comando de execução. Exemplo, sem valores:

```
cd Prototipos/backend
docker build -t ditec-api .
docker run --rm -p 8080:8080 \
  -e DITEC_PROFILE=prod \
  -e DB_URL="<URL-JDBC-DO-BANCO>" -e DB_USERNAME="<USUARIO>" -e DB_PASSWORD="<SENHA-
DO-BANCO>" \
  -e JWT_SECRET="<SEGREDO-COM-32-CARACTERES-OU-MAIS>" \
  -e FRONTEND_URL="<URL-PUBLICA-DO-FRONT-END>" \
  ditec-api
```

O `Dockerfile` não define `DITEC_PROFILE`; sem a variável, o perfil é `dev` (achado N-01, tratado na seção 6).

5 MIGRAÇÕES DO BANCO DE DADOS

O esquema é versionado pelo Flyway, que registra as migrações aplicadas na tabela `flyway_schema_history` e executa, em ordem, as que ainda não foram aplicadas. As migrações do projeto são duas:

a) `V1__schema.sql` cria as 9 tabelas, com 83 colunas, as chaves primárias e estrangeiras, as restrições CHECK e os índices;

b) `V2__rastreamento_os.sql` é incremental e preserva os dados: acrescenta 8 colunas (diagnóstico, orçamento, previsão e data de entrega em `ordem_servico`; tipo, usuário e visibilidade em `timeline_os`), amplia o domínio de status da OS para 10 valores, converte `EM_ATENDIMENTO` em `EM_MANUTENCAO` e cria dois índices.

O modelo resultante (91 colunas, 13 chaves estrangeiras, 11 restrições CHECK) está no Apêndice C. Foram identificadas três limitações da V2, com a respectiva orientação (achados N-03 a N-05 da revisão técnica, EQUIPE DITEC, 2026c).

5.1 Antes de aplicar a V2 em um banco com dados

1. **Versão do servidor:** MySQL 8.0.16 ou superior. A V2 usa `ALTER TABLE ... DROP CHECK`, sintaxe exclusiva do MySQL; no MariaDB a migração falha por erro de sintaxe (N-05).
2. **Cópia de segurança:** gerar um *dump* completo antes de migrar. O DDL do MySQL não é transacional, de modo que, se a V2 falhar no meio, as alterações já executadas permanecem e o Flyway registra a falha. A recuperação segura é restaurar a cópia, corrigir os dados e aplicar a V2 novamente (N-04).
3. **Pré-checagem de dados:** a V1 não impedia valores negativos em `ordem_servico.valor`, e a V2 cria a restrição `ck_os_valor` (`valor >= 0`). A consulta abaixo deve devolver `0`; caso contrário, os registros devem ser corrigidos antes (N-04).

```
SELECT COUNT(*) FROM ordem_servico WHERE valor < 0;
```

```
mysqldump --single-transaction ditec_assistencia > backup.sql
```

5.2 Roteiro de reversão da V2 corrigido

O roteiro de reversão do README do banco não removia os índices `idx_os_criado_em` e `idx_timeline_os_data`, e reaplicar a V2 depois da reversão falhava com erro de nome de índice duplicado (N-03). Esse defeito foi reproduzido e a correção foi validada em uma instância MariaDB de teste, com adaptação do comando `DROP CHECK` (42

verificações aprovadas, ver Apêndice F). O roteiro corrigido é o do README, acrescido de duas linhas antes da última instrução:

```
-- ... instruções do README, até o ADD CONSTRAINT / DROP COLUMN de ordem_servico ...
DROP INDEX idx_os_criado_em ON ordem_servico;
DROP INDEX idx_timeline_os_data ON timeline_os;
DELETE FROM flyway_schema_history WHERE version = '2';
```

A reversão apaga os dados das colunas criadas pela V2. A correção do README consta de um patch de documentação preparado pelos autores (arquivo `patches/correcoes-documentacao-pos-gemini.patch`, que acompanha esta entrega) e **ainda não aplicado ao repositório**, por depender de autorização do grupo (P04 do Apêndice D).

6 IMPLANTAÇÃO EM PRODUÇÃO

6.1 Arquitetura de implantação

O front-end é publicado no GitHub Pages e a API é executada em contêiner Docker no Render, conectada a um banco MySQL (ver a figura de arquitetura do Apêndice C). A API publicada está configurada em `config.js` como `https://ditec-api.onrender.com`.

6.2 Publicação do front-end

O fluxo `deploy-pages.yml` é executado a cada *push* no ramo `main` (ou manualmente): faz o *checkout*, copia a pasta `Código-fonte` para uma pasta de publicação, remove os arquivos `.bak`, `README.md` e `SECURITY.md` e publica o resultado no GitHub Pages. O fluxo usa o segredo `PAGES_TOKEN` do repositório, cujo valor não consta do código. Se a URL da API mudar, é preciso atualizar `config.js` e a diretiva `connect-src` da política de segurança de conteúdo (`meta CSP`) das quatro páginas HTML.

6.3 Publicação da API

No Render, o serviço é um contêiner construído a partir do `Dockerfile` de `Prototipos/backend`, com as variáveis da Tabela 5 cadastradas no painel do serviço, e a rota `/actuator/health` pode ser usada como verificação de saúde da hospedagem. O repositório **não documenta** a configuração do serviço no Render nem o procedimento de criação do primeiro administrador em produção, já que o semeador fica desativado nesse perfil; esses pontos são a pendência P14 do Apêndice D.

6.4 Lista de verificação da configuração de produção

A configuração do serviço no Render **não pôde ser consultada** durante a elaboração deste documento. A Tabela 6 é a lista que o grupo deve percorrer no painel do serviço. O item mais importante é o primeiro (achado N-01): como o `Dockerfile` não define o perfil, um serviço sem a variável `DITEC_PROFILE=prod` inicia no perfil de desenvolvimento, com o semeador de dados de demonstração ativo e a regra de CORS de desenvolvimento.

Tabela 6 – Lista de verificação da configuração de produção

Item	Valor esperado	Por quê
`DITEC_PROFILE`	`prod`	O perfil de produção desativa o semeador, ativa o tratamento de cabeçalhos de proxy e reduz o nível de registro de eventos

Item	Valor esperado	Por quê
`DITEC_SEED_ENABLED`	`false`	Redundante no perfil `prod`, mas protege caso o perfil não seja aplicado; o semeador só roda se não houver administrador
`JWT_SECRET`	Valor aleatório de 32 caracteres ou mais, exclusivo da produção	Segredo de assinatura dos tokens
`DB_PASSWORD`	Senha própria, diferente da de desenvolvimento	Acesso ao banco
`FRONTEND_URL` ou `DITEC_ALLOWED_ORIGINS`	URL pública do front-end, sem barra final	CORS restrito à origem do site
`DITEC_TRUST_PROXY`	`true` (padrão do perfil `prod`)	O Render é um proxy confiável; sem isso, todas as requisições aparecem com o endereço do proxy e dividem o mesmo limite
`GEMINI_API_KEY`	Chave própria do projeto, nunca a de outra pessoa	Habilita a IA; deve estar apenas no painel do serviço
Saúde	`GET /actuator/health` responde `UP`	Confirma que a aplicação e o banco estão operantes
Contas de demonstração	Inexistentes	Verificar que não é possível entrar com as contas do README

Fonte: Elaborado pelos autores (2026), a partir de `application-prod.yml`, `Dockerfile` e da revisão técnica (EQUIPE DITEC, 2026c).

7 INTEGRAÇÃO CONTÍNUA (PROPOSTA)

A disciplina apresenta a integração e a entrega contínuas como práticas de manutenção (PROJETO E DESENVOLVIMENTO DE SOFTWARE, [s. d.]). O repositório tem o fluxo de publicação do front-end, mas não tem fluxo que compile e teste o back-end a cada alteração. O arquivo abaixo é uma **proposta**, a ser adicionada como `.github/workflows/ci.yml` **após a autorização do grupo**; ela não foi aplicada ao repositório nem executada. Ela usa o JDK 21 e o mesmo comando de testes da seção de execução.

```
name: CI do back-end
on:
  push:
    branches: [main]
  pull_request:
jobs:
  testes:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Prototipos/backend
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Compilar e executar os testes
        run: mvn -B test
```

Com esse fluxo e a proteção do ramo `main` exigindo a aprovação do fluxo e a revisão de um segundo integrante antes do *merge*, o critério h da definição de pronto (Apêndice D) passaria a ser atendido.

8 SOLUÇÃO DE PROBLEMAS

A Tabela 7 relaciona os problemas mais prováveis na execução e a verificação sugerida, com base no comportamento descrito no código e na documentação do projeto.

Tabela 7 – Problemas comuns na execução e verificações sugeridas

Sintoma	Causa provável	Verificação
A aplicação não inicia e reclama de senha ou segredo	`DB_PASSWORD` ou `JWT_SECRET` ausente (ou segredo com menos de 32 caracteres)	Conferir o arquivo `.env` e se o script o carregou
Erro de conexão ao banco	Banco ou usuário não criado, senha diferente, MySQL parado	Executar o script do Passo 2 e conferir `DB_URL`, `DB_USERNAME` e `DB_PASSWORD`
Erro de sintaxe na migração V2	Servidor MariaDB ou MySQL anterior à 8.0.16	Usar MySQL 8.0.16 ou superior
Falha na V2 por restrição de valor	OS com valor negativo preexistente	Executar a pré-checagem e corrigir os registros
O navegador bloqueia as chamadas à API (CORS)	Origem do front-end não liberada	Informar a origem em `DITEC_ALLOWED_ORIGINS` ou servir o front-end nas portas 5500 ou 5501
Resposta 429	Limite de requisições por origem excedido	Aguardar a janela do limite (ver a regra RN17 do Apêndice B)
O chatbot responde com texto de perguntas frequentes	Chave da IA ausente ou inválida, limite do plano atingido ou pausa temporária	Conferir `GEMINI_API_KEY` e `GEMINI_MODEL`; o modo automático é o comportamento previsto nesse caso
Login como administrador não funciona em produção	O semeador está desativado (comportamento correto em produção) e o repositório não documenta como criar o primeiro administrador	Pendência P14 do Apêndice D

Fonte: Elaborado pelos autores (2026), a partir do código e da documentação do projeto (EQUIPE DITEC, 2026a).

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

EQUIPE DITEC. **DITEC Assistência Técnica**: repositório do projeto (commit e58a3d2). [S. l.]: GitHub, 2026a. Disponível em: https://github.com/wikimiwi/DITEC-Landing-Page-main--v2. Acesso em: 9 nov. 2026.

EQUIPE DITEC. **Relatório final de correções — DITEC (09/11/2026b)**. [S. l.]: [s. n.], 2026b. Documento versionado no repositório (docs/RELATORIO_FINAL_CORRECOES_DITEC.md).

EQUIPE DITEC. **Revisão técnica do código atualizado — projeto DITEC Assistência Técnica**. [S. l.]: [s. n.], 2026c. Documento complementar (REVISAO_CODIGO_ATUALIZADO_09-11-2026c).

ORACLE CORPORATION. **MySQL 8.0 reference manual**. [S. l.]: Oracle, [s. d.]. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 9 nov. 2026.

PROJETO E DESENVOLVIMENTO DE SOFTWARE. **Da identificação do problema à entrega de uma solução funcional**: apresentação da disciplina Análise e Projeto de Sistemas II. [S. l.: s. n.], [s. d.]. 26 slides. Arquivo PPTX.

REDGATE SOFTWARE. **Flyway documentation**. Cambridge: Redgate Software, [s. d.]. Disponível em: https://documentation.red-gate.com/flyway. Acesso em: 9 nov. 2026.

SPRING. **Spring Boot reference documentation**. [S. l.]: Broadcom, [s. d.]. Disponível em: https://docs.spring.io/spring-boot/. Acesso em: 9 nov. 2026.